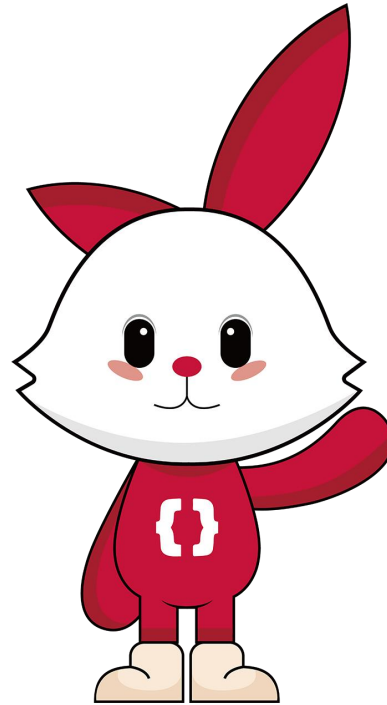




TSCINBUNY
ZYC0208



教程简介

本教程包括如下图所示的 3 份文件内容帮助你深入学习电子和编程知识：

“1_Get_start”：该文件夹存放机器人组装指南和必要的软件环境文件等，为了准确且快速的完成组装，请务必详细查看并按照手册进行组装；

1_Get_start-开始	2025/1/17 17:39	文件夹
2_Code-代码文件	2025/1/17 17:39	文件夹
3_APP	2025/1/18 10:02	文件夹

“2_Code”：该文件夹用来存放代码文件，每份代码文件独立上传使用；

“3_APP”：该文件夹存放 APP 安装包；



目录

教程简介	2
1 蓝牙控制代码烧录	5
2 蓝牙遥控 APP	9
2.1 安装和设置 APP	9
3 第一个程序代码-Blink	11
3.1 关于本课:	11
3.2 主控板	11
3.3 拓展板	12
3.4 制作属于自己的“Blink”草图	13
4 电机测试	18
4.1 课程介绍	18
4.2 L293D 电机驱动芯片如何工作	18
4.3 接线图	19
4.4 代码分析	20
5 跟随小车	22
5.1 课程介绍	22
5.2 组件介绍	22
5.3 超声波是如何工作的?	23
5.4 跟随小车原理	24
5.5 代码分析	24
5.6 接线图	26
6 避障小车	27
6.1 课程介绍	27
6.2 避障原理	27
6.3 代码上传	27
7 循迹小车	29
7.1 三路循迹模块	29



7.2 循迹模块接线图	30
7.3 代码分析	31
8 舵机机械臂	34
8.1 课程介绍	34
8.2 舵机介绍	34
8.3 代码分析	35
9 综合控制	39
9.1 课程介绍	39
9.2 蓝牙综合控制代码分析	39



1 蓝牙控制代码烧录

确保已按照 1_Get_start 文件夹中指导完成编程环境的创建。

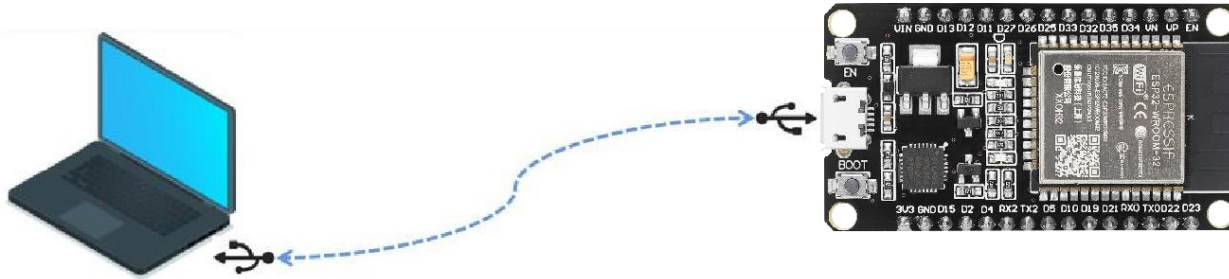
1_Get_start-开始	2025/1/17 17:39	文件夹
2_Code-代码文件	2025/1/17 17:39	文件夹
3_APP	2025/1/18 10:02	文件夹

打开代码文件（文件夹路径：\2_Arduino_Code-代码文件\0_ESP32-arm\0_ESP32-arm.ino）

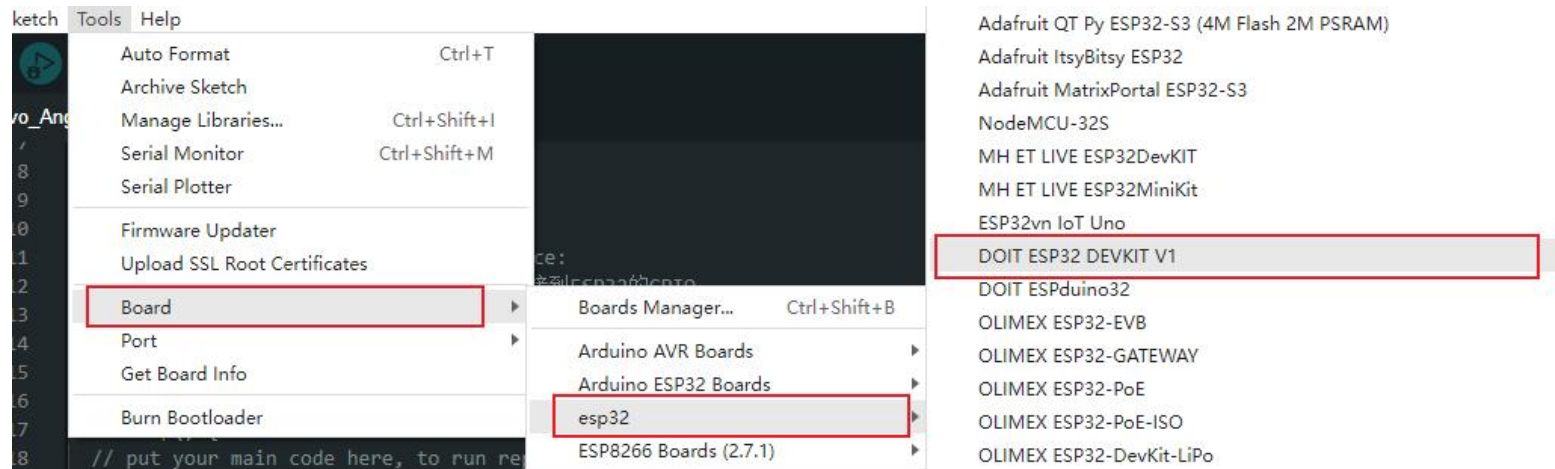
0_ESP32-arm	2024/11/30 17:27	文件夹
1_Motor_test	2024/11/28 16:35	文件夹
2_Ultrasonic_follow	2024/11/30 17:22	文件夹
3_Ultrasonic_obstacle_avoidance	2024/11/30 17:22	文件夹
4_Infrared_patrol_line	2024/11/30 17:22	文件夹
5_Robot_arm	2024/11/30 17:21	文件夹



用 USB 线将主板连接到电脑

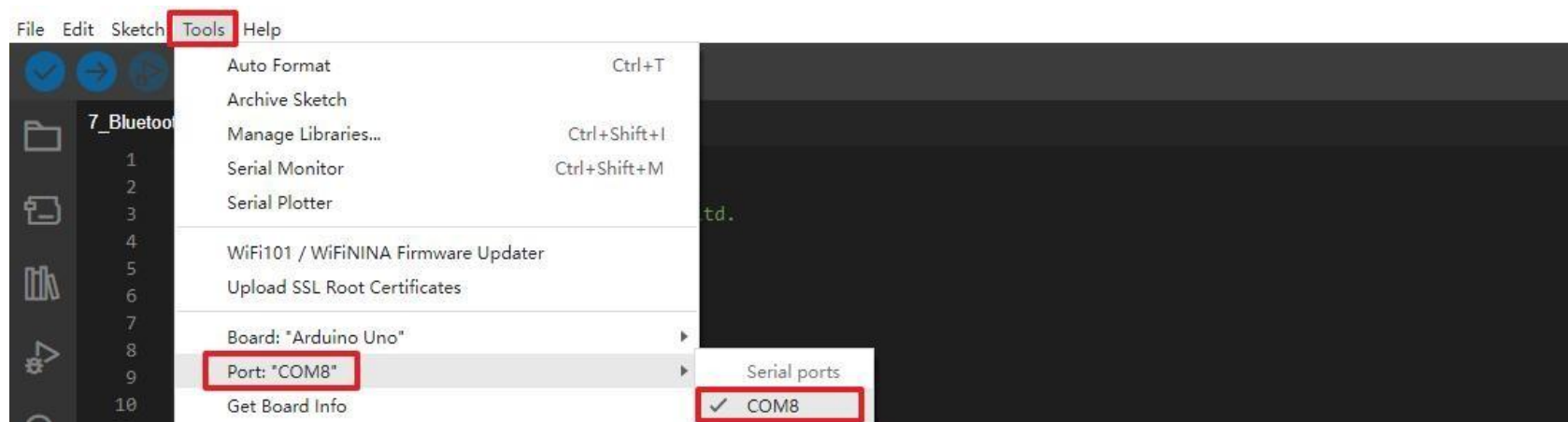


打开“工具”菜单选项，点击“开发板”，选择板类型 DOIT ESP32 DEVKIT V1。

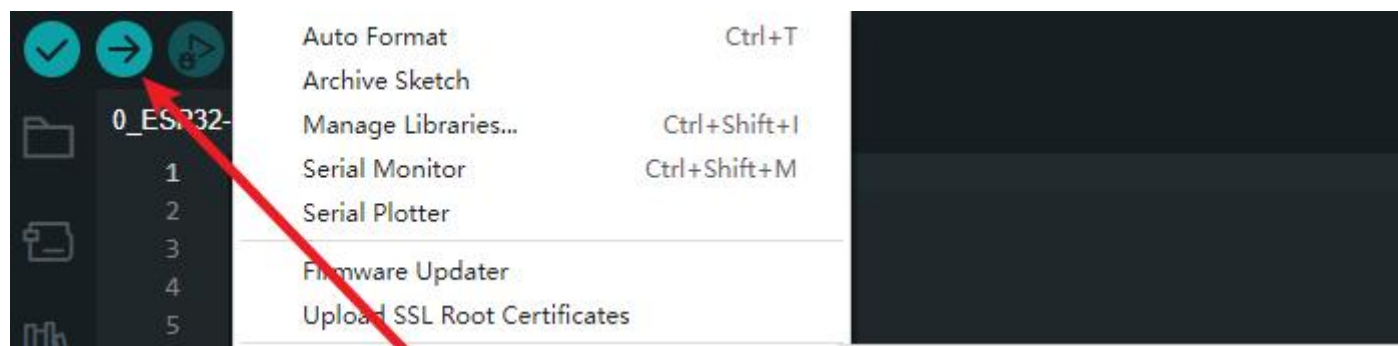




再选择串口 COM 号，在这里是 COM8，但实际每个人的 COM 号会不一样。



点击“upload”按钮进行代码编译及上传。



在编辑器底部能看到上传写入完成 100%，如果出现错误或失败，请检查板类型和串口号是否正确。



```
Writing | ##### | 100% 0.83s  
avrdude: 5418 bytes of flash written  
avrdude done. Thank you.
```

Done uploading.

Ln 18, Col 27 Arduino Uno on COM8 3

接下来开始蓝牙 APP 的安装和设置，请看后面一节。



2 蓝牙遥控 APP

2.1 安装和设置 APP

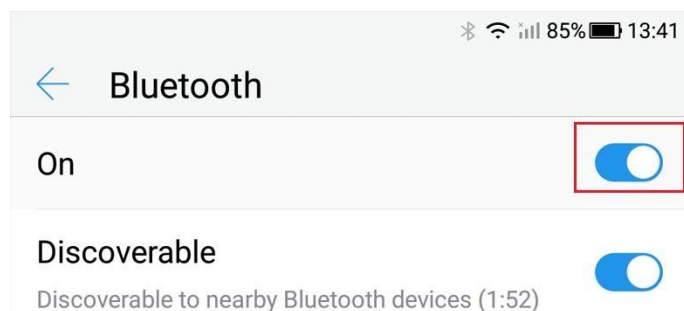
打开目录 3_APP，将” TSCINBUNY. apk” 安装到安卓手机并打开.

名称	修改日期	类型	大小
 TSCINBUNY.apk	2025/1/18 10:01	APK 文件	40,912 KB

ios 设备打开应用商店 App Store，搜索并安装 TSCINBUNY



接下来，我们使用 Android 手机演示如何通过这个应用程序控制汽车：首先，打开手机的蓝牙功能开关





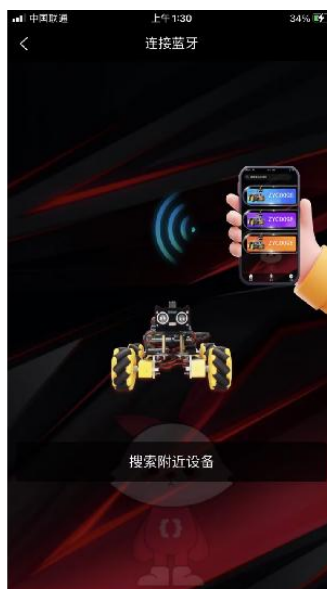
打开刚才安装好的 APP 并确保机器人小车已经通电。



选择小车型号



控制主界面



搜索附近设备

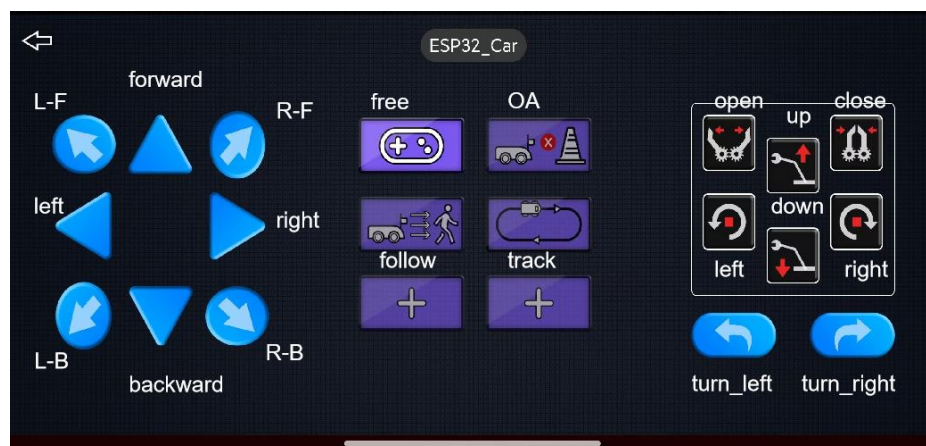


连接设备



操作

之后会出现小车的蓝牙名称 默认为 ESP_32car 点击操作按钮。



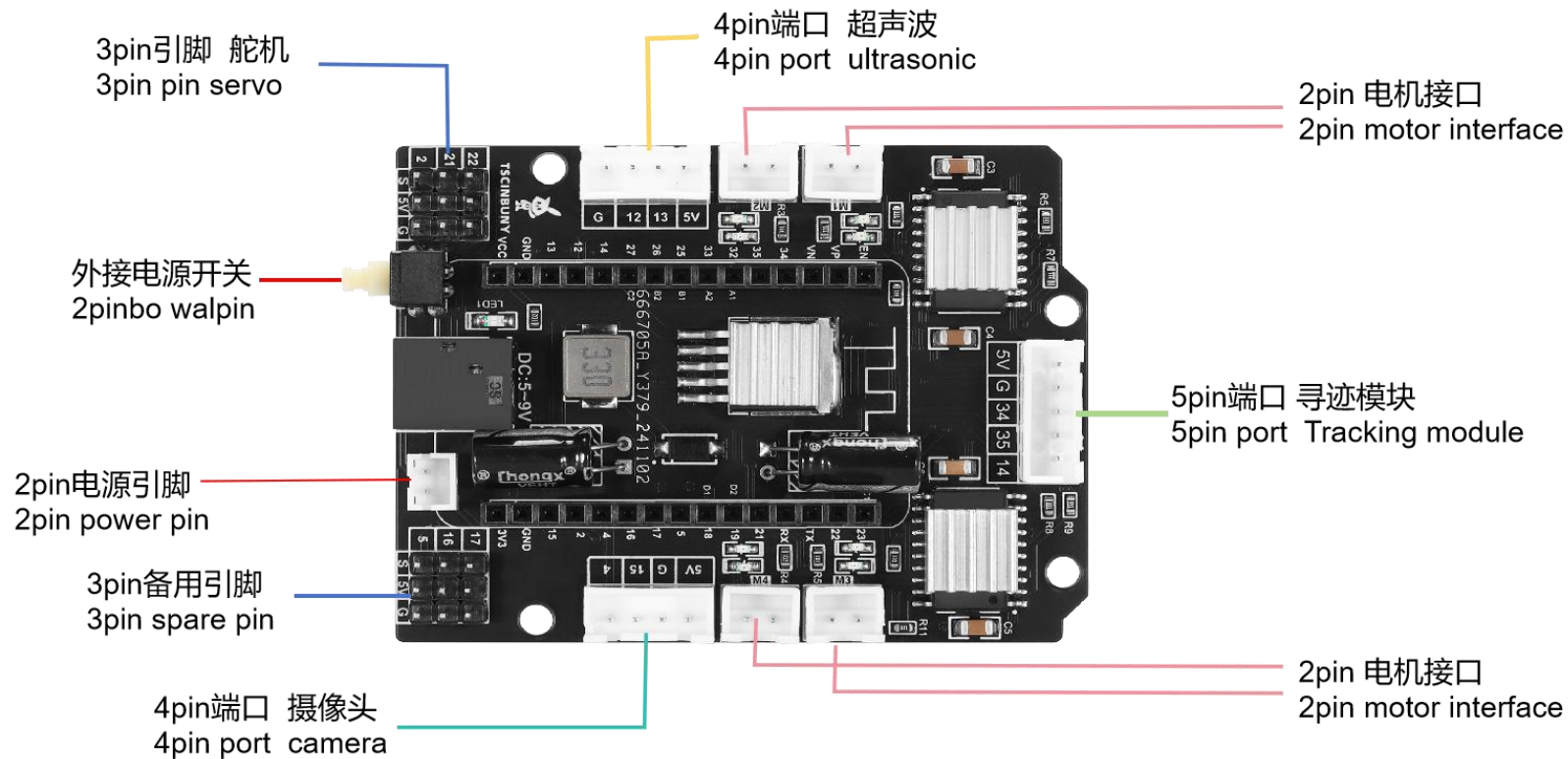
后面的课程将详细介绍并指导你学习本套件！

3 第一个程序代码-Blink



当您用 USB 线将主板连接到电脑时,您可能会发现它的 LED 已经闪烁,这是因为板子预装了“Blink”草图。

3.3 拓展板





3.4 制作属于自己的“Blink”草图

在本课中,我们将使用我们自己的 Blink 草图重新给主板编程,然后更改它的闪烁速率。现在保持将主板连接到电脑,设置 Arduino IDE 并确保您可以找到正确的串行端口,并上传程序进行测试。

Arduino IDE 包含大量您可以加载和使用的示例草图,其中包括一个用于制作“LED”的“眨眼”示例草图。在 IDE 中的菜单系统 File > Examples > 01.Basics 中您将找到的“Blink”草图。

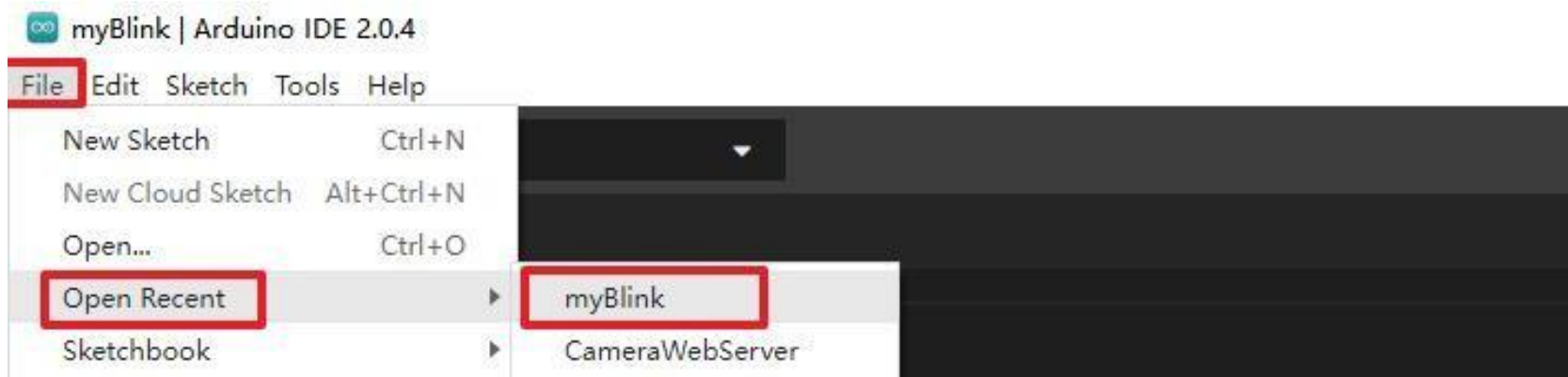


Arduino IDE 中包含的示例草图是“只读的”。也就是说,您可以将它们上传到主控板上,但如果您更改它们,您不能将它们保存为同一个文件。因此您需要做的第一件事是保存自己的副本。

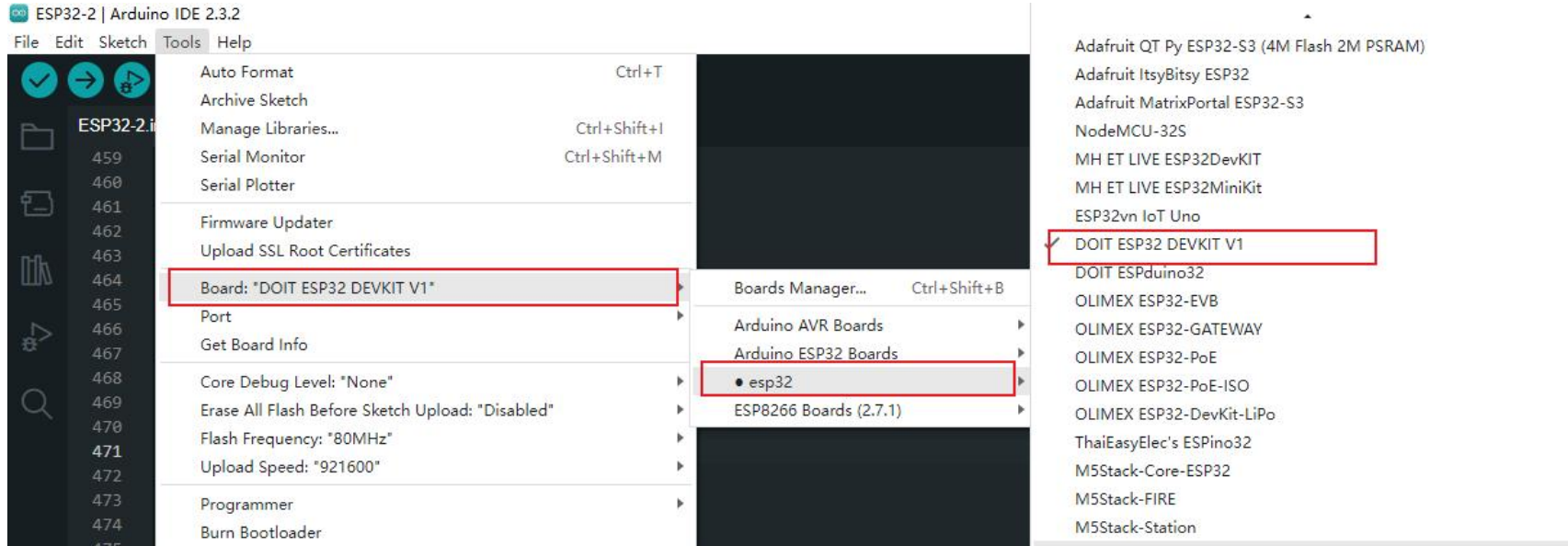
从 Arduino IDE 的文件菜单中,选择“另存为..”,然后将草图保存为“MyBlink”。

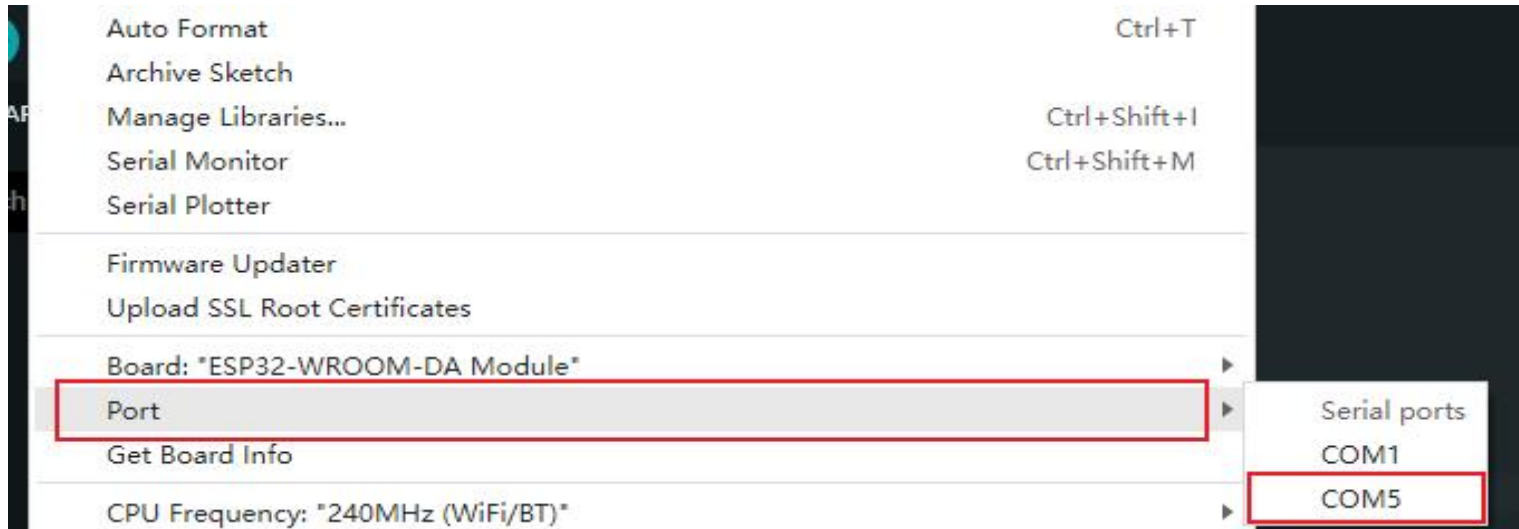


您已将“闪烁”副本保存在速写本中,如果您想再次找到它,只需使用“File >Open Recent”菜单选项打开它。



使用 USB 连接线将 ESP32 板连接到计算机并检查 Board Type 和 Serial Port 设置正确

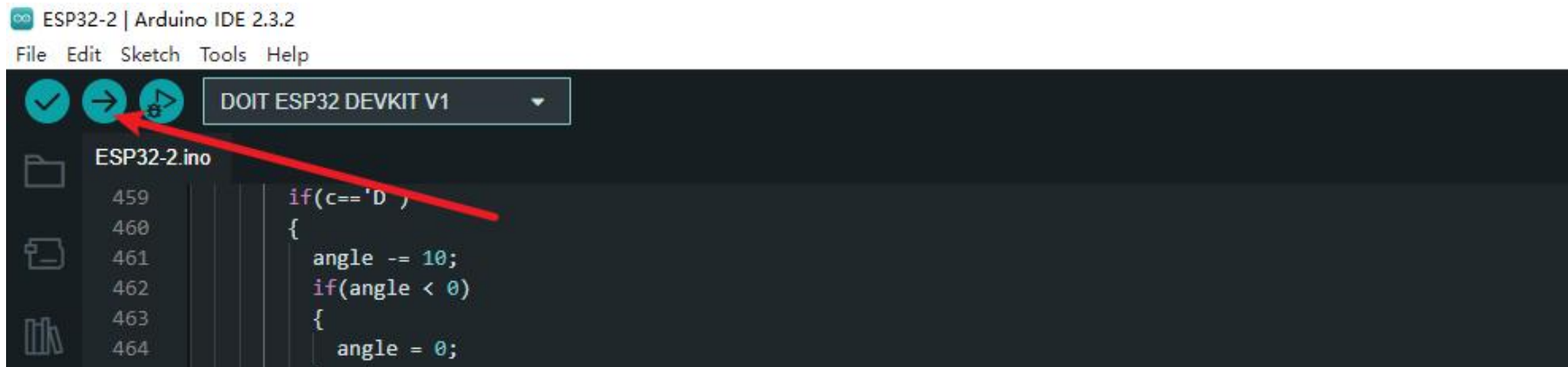




注意:这里的板卡类型选择 ESP32 DEVKIT V1 和串口选择 COM5。

实际上串行端口显示为每个人都不一样, 尽管这里选择了 COM5, 但它可能是您计算机上的 COM3 或 COM4。一个正确的 COM 端口应该是符合 COM X 标准的。

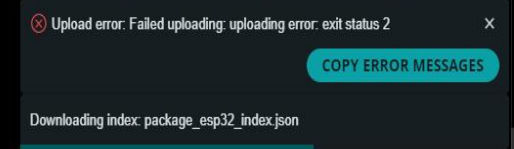
点击“上传”按钮后程序开始上传, 此时, ESP32 上的 LED 会随着草图传输而开始闪烁。





```
Using library ESP32Servo at version 3.0.5 in folder: E:\工作\课程设计\Arduino_file\libraries\ESP32Servo
Using library BluetoothSerial at version 2.0.0 in folder: C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\hardware\esp32\2.0.12\libraries\BluetoothSerial
"C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\tools\xtensa-esp32-elf-gcc\esp-2021r2-patch5-8.4.0\bin\xtensa-esp32-elf-size" -A "C:\Users\Administrator\AppData\Local\Temp\arduino\sketches\9B54E0FB643161932
Sketch uses 1060849 bytes (80%) of program storage space. Maximum is 1310720 bytes.
Global variables use 39060 bytes (11%) of dynamic memory, leaving 288620 bytes for local variables. Maximum is 327680 bytes.
"C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\tools\esptool_py\4.5.1/esptool.exe" --chip esp32 --port "COM1" --baud 921600 --before default_reset --after hard_reset write_flash -z --flash_mode dio --flash_freq 80m --flas
esptool.py v4.5.1
Serial port COM1
Connecting.....

A fatal error occurred: Failed to connect to ESP32: No serial data received.
For troubleshooting steps visit: https://docs.espressif.com/projects/esptool/en/latest/troubleshooting.html
Failed uploading: uploading error: exit status 2
```



在“编译草图..”过程中，您也许可能会收到错误消息：

这可能意味着您的板根本没有连接，或者 CP210X 驱动程序没有自动安装（如果需要）或者串行端口选择错误。如果您遇到这种情况，请检查您的 IDE 设置和主板连接情况，截屏后寻求业务人员帮助。或者请到官网自行下载驱动 下载网站如下：

<https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers>

上传完成后，电路板 LED 应重新启动并开始闪烁。请注意，此草图的很大一部分是由注释组成的。这些不是实际的程序指令；相反，他们只是解释如何让该程序有效。他们在那里是为了您的易于阅读。草图顶部“/*”和“*/”之间的所有内容都是块注释，它解释了草图的用途。

单行注释以“//”开头，直到该行末尾的所有内容都被视为注释。

第一部分代码是：



```
25 // the setup function runs once when you press reset or power the board
26 void setup() {
27     // initialize digital pin LED_BUILTIN as an output.
28     pinMode(LED_BUILTIN, OUTPUT);
29 }
30
```

每个草图都需要“设置”功能，也就是“Void setup()”函数，这是在按下重置按钮时执行的。每当电路板因任何原因而重置时，例如首次通电或上传草图后它就被执行。

接下来是给引脚命名和设置输出，这里设置“LED_BUILTIN”为输出端口。在大多数 ESP32 上, 包括 DEVKIT V1, 2 引脚是 LED 对应的引脚，而为了方便编程，程序已经将 LED_BUILTIN 变量设置为该引脚，所以直接使用不需要重新命名为 2 引脚。

草图还必须具有“循环”功能。与只运行一次的“设置”功能不同, 在重置后，“循环”功能将在完成命令运行后立即重新开始。

```
// the loop function runs over and over again forever
void loop() {
    digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is the voltage level)
    delay(1000);                        // wait for a second
    digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by making the voltage LOW
    delay(1000);                        // wait for a second
}
```

在循环函数内部, 命令首先打开 LED 引脚(高电平), 然后“延迟 1000 毫秒(1 秒)”, 然后关闭 LED 引脚并暂停一秒。你现在要让你的 LED 闪烁得更快。正如您可能已经猜到的那样, 关键在于更改“delay()”中的参数。



```
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is the voltage level)  
  delay(1000);                        // wait for a second  
  digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by making the voltage LOW  
  delay(1000);                        // wait for a second  
}
```

此延迟时间以毫秒为单位，因此如果您希望“LED”以两倍的速度闪烁，请将值从“1000”更改为“500”。这将在每次延迟时暂停半秒，而不是一秒。再次上传草图，您应该会看到“LED”开始更快地闪烁。

至此，您已经了解和掌握了基本的 Arduino 编程知识和下载程序的基本步骤，这为后面学习复杂项目打下很好的基础。

注：有些时候下载程序不成功的原因是电压问题或者主控板没有进入下载模式，这个时候需要开启电池盒电源以及在烧录程序的时候连续电机 Boot 按钮使主控板进入下载模式。

4 电机测试

4.1 课程介绍

上一节课中我们学会了基本的上传程序方法，本节课将学习并掌握汽车移动的控制原理。

4.2 L293D 电机驱动芯片如何工作

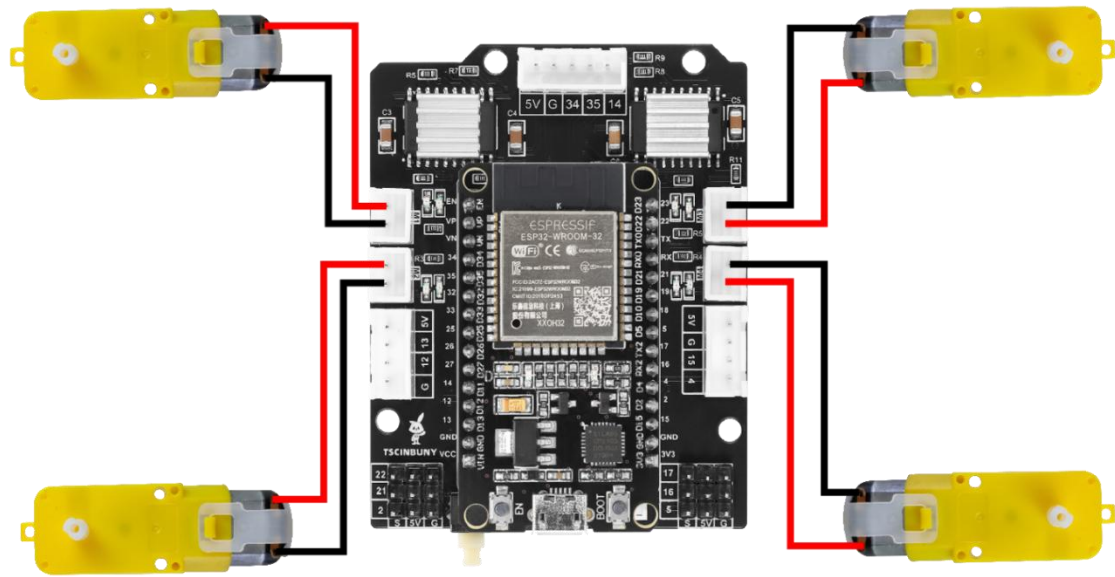
L293D 是通过控制输入端口（IN1、IN2、IN3、IN4）的高低电平来决定电机的转动方向，配合使能信号（EN1/2、EN3/4）



实现电机的启动和停止。

EN1/2	IN1	IN2	电机状态
高	高	低	正转
高	低	高	反转
高	高	高	刹车（快速停止）
高	低	低	自由停止
低	X	X	停止

4.3 接线图





4.4 代码分析

打开代码文件（路径：\2_Code-代码文件\1_Motor_test\1_Motor_test.ino）

定义电机引脚：

```
1 //右
2 #define RightFrontFWD 23
3 #define RightFrontBWD 27
4 #define RightBackFWD 33
5 #define RightBackBWD 32
6
7 //左
8 #define LeftFrontFWD 18
9 #define LeftFrontBWD 19
10 #define LeftBackFWD 26
11 #define LeftBackBWD 25
```

定义电机引脚模式：

```
void setup() {
    // put your setup code here, to run once:
    pinMode(RightFrontFWD, OUTPUT);
    pinMode(RightFrontBWD, OUTPUT);
    pinMode(RightBackFWD, OUTPUT);
    pinMode(RightBackBWD, OUTPUT);
    pinMode(LeftFrontFWD, OUTPUT);
    pinMode(LeftFrontBWD, OUTPUT);
    pinMode(LeftBackFWD, OUTPUT);
    pinMode(LeftBackBWD, OUTPUT);
}
```



循环函数 `loop()`，重复执行前进、后退、左转和右转各 3 秒，当然，你也可以修改 2000 为 4000 看看效果如何

```
void loop() {  
  digitalWrite(RightFrontBWD, motorSpeed); // 右前，前转，使用PWM控制速度  
  digitalWrite(RightFrontFWD, 0);          // 如果RightFrontFWD不是PWM引脚，则设为低电平  
  digitalWrite(RightBackBWD, 0);           // 右后，前转，使用PWM控制速度  
  digitalWrite(RightBackFWD, motorSpeed);  // 如果RightBackBWD不是PWM引脚，则设为低电平  
  
  digitalWrite(LeftFrontFWD, motorSpeed);  // 左前，前转，使用PWM控制速度  
  digitalWrite(LeftFrontBWD, 0);           // 如果LeftFrontBWD不是PWM引脚，则设为低电平  
  digitalWrite(LeftBackBWD, 0);           // 左后，前转，使用PWM控制速度  
  digitalWrite(LeftBackFWD, motorSpeed);   // 如果LeftBackBWD不是PWM引脚，则设为低电平  
  delay(3000);  
  
  digitalWrite(RightFrontFWD, motorSpeed); // 右前轮，后退方向，利用PWM进行速度调控  
  digitalWrite(RightFrontBWD, 0);          // 若RightFrontBWD不支持PWM，则设为低电平  
  digitalWrite(RightBackFWD, 0);          // 右后轮，后退方向，利用PWM进行速度调控  
  digitalWrite(RightBackBWD, motorSpeed);  // 若RightBackFWD不支持PWM，则设为低电平  
  
  digitalWrite(LeftFrontBWD, motorSpeed);  // 左前轮，后退方向，利用PWM进行速度调控  
  digitalWrite(LeftFrontFWD, 0);           // 若LeftFrontFWD不支持PWM，则设为低电平  
  digitalWrite(LeftBackFWD, 0);           // 左后轮，后退方向，利用PWM进行速度调控  
  digitalWrite(LeftBackBWD, motorSpeed);   // 若LeftBackFWD不支持PWM，则设为低电平  
  delay(3000);  
  
  digitalWrite(RightFrontBWD, motorSpeed); // 右前轮，前退方向，利用PWM进行速度调控  
  digitalWrite(RightFrontFWD, LOW);        // 若RightFrontFWD不支持PWM，则设为低电平  
  digitalWrite(RightBackFWD, motorSpeed);  // 右后轮，前进方向，利用PWM进行速度调控  
  digitalWrite(RightBackBWD, LOW);        // 若RightBackBWD不支持PWM，则设为低电平  
  
  digitalWrite(LeftFrontBWD, motorSpeed);  // 左前轮，前进方向，利用PWM进行速度调控  
  digitalWrite(LeftFrontFWD, LOW);         // 若LeftFrontBWD不支持PWM，则设为低电平  
  digitalWrite(LeftBackBWD, motorSpeed);   // 左后轮，前进方向，利用PWM进行速度调控  
  digitalWrite(LeftBackFWD, LOW);         // 若LeftBackFWD不支持PWM，则设为低电平  
  delay(3000);  
}
```




5 跟随小车

5.1 课程介绍

超声波测距是一种非常有用且广泛使用的测量方式，本节课主要了解超声波模块工作原理、掌握超声波电路图的连接和通过编程学会超声波模块测距的方法。

5.2 组件介绍

HC-SR04 超声波模块：



Working voltage	DC-5V
Working current	15mA
Working frequency	40KHz
Distance range	2cm~400cm



Measuring angle	15°
Input trigger signal	10 US TTL pulse
Output echo signal	Output TTL level signal, proportional to the range

5.3 超声波是如何工作的？

1. 发射器（trig pin）发送信号：高频声音；
2. 当信号碰到物体时，它会被反射；
3. 接收器（echo pin）：接收从它反射回来的信号。

超声波测距的方法有很多种，该系统在超声波测量中的原理是：检测来自超声波发射器的超声波通过气体介质到接收器的传输时间，将此时间乘以空气中的声速，得出声音传播的距离。

超声波发射器向一定方向发射超声波，同时 MCU 开始计时，超声波在空中发射，途中遇到障碍物立即返回，超声波接收器接收到反射波立即停止计时。

根据定时器记录的时间 T ，可以计算出发射点到障碍物的距离（s）。

$$\text{公式： } S = VT/2$$

四个因素限制了超声波系统的最大可测量距离：超声波的幅度、反射器的纹理、反射和入射声波之间的角度以及接收换能器的灵敏度。接收换能器直接接收声脉冲的能力将决定最小可测量距离。



5.4 跟随小车原理

判断正前方超声波检测距离大于 25cm 就跟随前进，距离在 15cm ~25cm 之间就停止，最后在但超声波距离过近小于 15cm 时汽车就后退。

5.5 代码分析

打开代码文件（路径：\2_Code-代码文件\2_Ultrasonic_follow\2_Ultrasonic_follow.ino）

定义超声波传感器引脚

```
//超声波  
#define Trigpin 13  
#define Echopin 12
```

初始化超声波传感器与电机引脚。

```
pinMode(Trigpin,OUTPUT);  
pinMode(Echopin,INPUT);  
  
pinMode(RightFrontFWD, OUTPUT);  
pinMode(RightFrontBWD, OUTPUT);  
  
pinMode(LeftFrontFWD, OUTPUT);  
pinMode(LeftFrontBWD, OUTPUT);
```



超声波测距函数

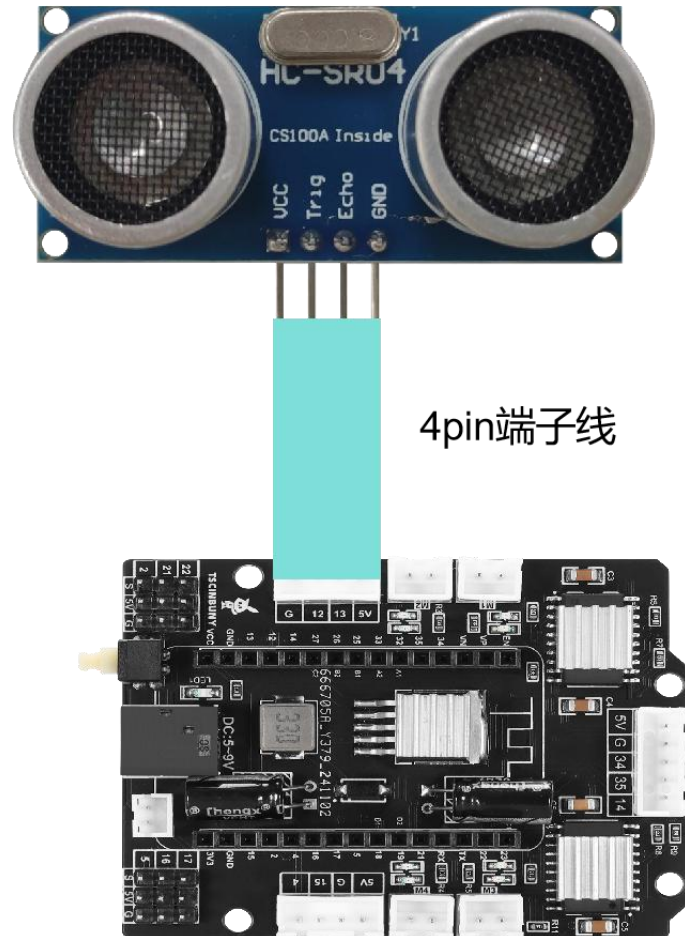
```
unsigned long measureDistance() {  
    digitalWrite(Trigpin, HIGH);  
    delayMicroseconds(20);  
    digitalWrite(Trigpin, LOW);  
    delayMicroseconds(10);  
    unsigned long Time_Echo_us = pulseIn(Echopin, HIGH, 60000); // 设置超时时间为60000微秒  
    if(Time_Echo_us > 1) {  
        return (Time_Echo_us * 34 / 100) / 2;  
    } else {  
        return 0; // 如果读取失败, 返回0  
    }  
}
```

小车运动函数（已折叠）

```
void moveForward(int motorSpeed) {  
    //前进  
    digitalWrite(RightFrontBWD, motorSpeed); // 右前, 前转, 使用PWM控制速度  
    digitalWrite(RightFrontFWD, 0);           // 如果RightFrontFWD不是PWM引脚, 则设为低电平  
    digitalWrite(RightBackBWD, 0);           // 右后, 前转, 使用PWM控制速度  
    digitalWrite(RightBackFWD, motorSpeed);   // 如果RightBackBWD不是PWM引脚, 则设为低电平  
  
    digitalWrite(LeftFrontFWD, motorSpeed);   // 左前, 前转, 使用PWM控制速度  
    digitalWrite(LeftFrontBWD, 0);            // 如果LeftFrontBWD不是PWM引脚, 则设为低电平  
    digitalWrite(LeftBackBWD, 0);            // 左后, 前转, 使用PWM控制速度  
    digitalWrite(LeftBackFWD, motorSpeed);    // 如果LeftBackBWD不是PWM引脚, 则设为低电平  
}  
  
void moveBackward(int motorSpeed) { ...  
}  
  
unsigned long measureDistance() { ...  
}
```



5.6 接线图





6 避障小车

6.1 课程介绍

在上节课程中我们已经了解和学习了超声波传感器的相关知识以及跟随小车的工作原理，本次课程我们来完成一辆自动避障的汽车。

6.2 避障原理

小车超声波检测距离前方小于等于 15cm（也就是遇到障碍时），小车后退后左转以此避开前方的障碍物。

6.3 代码上传

打开代码文件（路径：`\2_Code-代码文件\Ultrasonic_obstacle_avoidance\Ultrasonic_obstacle_avoidance.ino`）

代码中传感器引脚和获取超声波距离函数与上节课一致，移动函数增加一个左转函数。



```
void rotateLeft(int motorSpeed) {  
    //左转  
    digitalWrite(RightFrontBWD, motorSpeed); // 右前轮, 前退方向, 利用PWM进行速度调控  
    digitalWrite(RightFrontFWD, LOW); // 若RightFrontFWD不支持PWM, 则设为低电平  
    digitalWrite(RightBackFWD, motorSpeed); // 右后轮, 前进方向, 利用PWM进行速度调控  
    digitalWrite(RightBackBWD, LOW); // 若RightBackBWD不支持PWM, 则设为低电平  
  
    digitalWrite(LeftFrontBWD, motorSpeed); // 左前轮, 后退方向, 利用PWM进行速度调控  
    digitalWrite(LeftFrontFWD, LOW); // 若LeftFrontBWD不支持PWM, 则设为低电平  
    digitalWrite(LeftBackBWD, motorSpeed); // 左后轮, 后退方向, 利用PWM进行速度调控  
    digitalWrite(LeftBackFWD, LOW); // 若LeftBackFWD不支持PWM, 则设为低电平  
}
```

当障碍物的位置距离小车小于 15cm 时, 小车急停, 接着后退, 最后左转避开障碍物。

```
void loop() {  
    unsigned long Len_mm = measureDistance();  
    Serial.print(Len_mm, DEC);  
    Serial.println("mm");  
    delay(100);  
    if(Len_mm >= 150) moveForward(motorSpeed);  
    if(Len_mm < 150)  
    {  
        stopMoving();  
        Serial.print("Len_mm: ");  
        Serial.println(Len_mm);  
  
        moveBackward(motorSpeed); //后退  
        delay(800);  
        rotateLeft(motorSpeed); //左移  
        delay(1500);  
        stopMoving();  
    }  
}
```



7 循迹小车

7.1 三路循迹模块

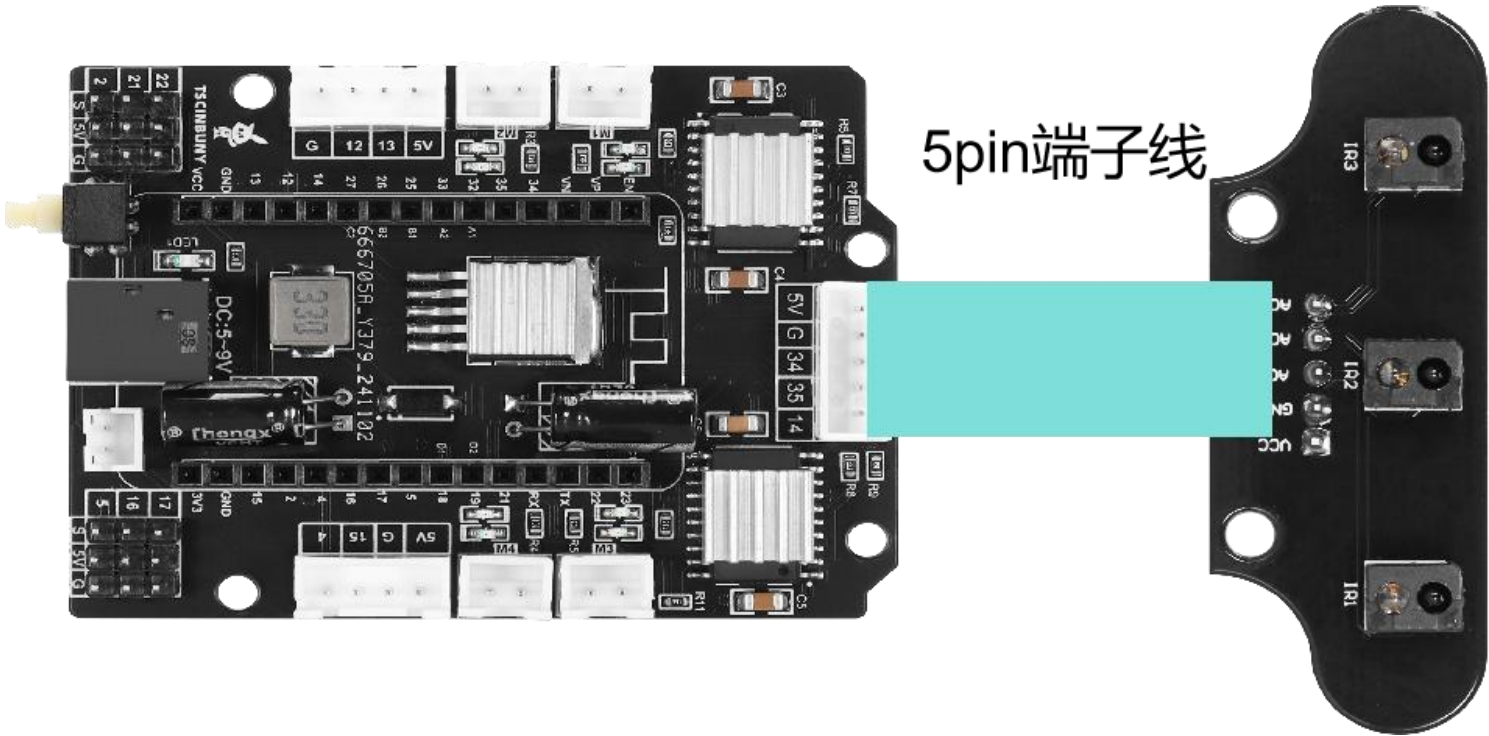


三通道跟踪模块如何工作？

该传感器属于红外反射检测，即利用不同的红外光物体表面的颜色具有不同的反射强度特点，在汽车行走过程中不断发射红外线光到地面，当红外光遇到白色地板时漫反射，反射光被安装在车内的接收管接收；如果遇到黑线，红外光是被吸收，车上的接收管没有接收到红外线。因此可通过是否接收反射红外光为依据确定黑线的位置和汽车行驶路线。



7.2 循迹模块接线图





7.3 代码分析

打开代码文件（路径：2_Code-代码文件\4_Infrared_patrol_line\4_Infrared_patrol_line.ino）

定义三个循迹传感器变量和设置引脚为输入

```
Infrared_patrol_line.ino
1  #define LEFT_LINE_TRACJING    34
2  #define CENTER_LINE_TRACJING  35
3  #define right_LINE_TRACJING   14
4  //右
```

```
63  Serial.begin(115200);
64  pinMode(LEFT_LINE_TRACJING, INPUT);
65  pinMode(CENTER_LINE_TRACJING, INPUT);
66  pinMode(right_LINE_TRACJING, INPUT);
67
```

循环函数体内获取 3 路循迹模块检测信息，并根据检测结果进行汽车运动控制（高电平为检测到黑线）。

在循环函数 loop() 中，将读取到的三路传感器的数值保存到变量中



```
void loop() {  
  // put your main code here, to run repeatedly:  
  Left_Tra_Value = analogRead(LEFT_LINE_TRACJING);  
  Center_Tra_Value = analogRead(CENTER_LINE_TRACJING);  
  Right_Tra_Value = analogRead(right_LINE_TRACJING);  
}
```

代码上传后我们打开串口监视器查看三个红外传感器读取的数值，正常情况检测到黑线的传感器读取数值为 3500 以上，当我们看到下面数值情况说明 LEFT 传感器检测到黑线，且其他传感器没有检测到黑线。

Serial Monitor ×

Not connected. Select a board and a port to connect automatically.

Both NL & CR ▼ 115200

Left: 4095	Center: 150	Right: 126
Left: 4095	Center: 151	Right: 124
Left: 4095	Center: 150	Right: 126
Left: 4095	Center: 147	Right: 121
Left: 4095	Center: 144	Right: 121
Left: 4095	Center: 144	Right: 122
Left: 4095	Center: 146	Right: 122
Left: 4095	Center: 150	Right: 121



将小车放置到黑色路线行走控制小车实现循迹功能的代码如下：

```
if (Left_Tra_Value < Black_Line && Right_Tra_Value >= Black_Line && Right_Tra_Value < Black_Line)
{
    int motorSpeed = 1;
    moveForward(motorSpeed);
}

if (Left_Tra_Value >= Black_Line && Left_Tra_Value > Right_Tra_Value)
{
    rotateLeft(motorSpeed); //左移
}

if (Right_Tra_Value >= Black_Line && Right_Tra_Value > Left_Tra_Value)
{
    rotateRight(motorSpeed); //右移
}

if (Left_Tra_Value >= Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value >= Black_Line)
{
    stopMoving();
}

if (Left_Tra_Value < Black_Line && Center_Tra_Value < Black_Line && Right_Tra_Value < Black_Line)
{
    stopMoving();
}

delay(100);
}
```



8 舵机机械臂

8.1 课程介绍

本节内容主要认识舵机属性、特点和学习舵机相关知识，并掌握舵机调试方法和电路连接，最后在 Arduino 编程中体会舵机的工作方式。

8.2 舵机介绍



MG90S 伺服电机控制脉冲信号周期为 20MS 脉冲宽度调制信号（PWM），脉冲宽度从 0.5ms 到 2.5ms，对应的转向位置从 0 到 180 度，线性变化。

也就是说，如果给舵机提供一定的脉冲宽度，它的输出轴就会保持一定的相应角度。无论外部扭矩如何变化，它都不会改变输出角度到一个新的对应位置，直到另一个脉冲信号提供给它。



舵机内部有参考电路，产生周期为 20ms、宽度为 1.5ms 的脉冲信号。有比较器，将外部信号与参考信号进行比较，判断方向和大小，从而产生电机转动信号。

8.3 代码分析

打开代码文件（路径：2_Code\Servo_Angle\Servo_Angle.ino）

导入 ESP32Servo 库文件，声明三个伺服电机连接的信号端口分别为 2/21/22

```
1  #include <ESP32Servo.h>
2
3  Servo myServo; // 创建一个Servo对象
4  Servo myServo1; // 创建一个Servo对象
5  Servo myServo2; // 创建一个Servo对象
6
```

```
10 void setup() {
11     // put your setup code here, to run once:
12     myServo.attach(2); // 将舵机的信号线连接到ESP32的GPIO
13     myServo1.attach(21); // 将舵机的信号线连接到ESP32的GPIO
14     myServo2.attach(22); // 将舵机的信号线连接到ESP32的GPIO
15 }
```



循环执行，转盘伺服电机从 140° 至 10° 往复旋转。

```
19     for ( angle = 140; angle >= 10; angle -= 1)
20     {
21         myServo.write(angle);
22         delay(15);
23     }
24     for (angle = 10 ; angle<= 140; angle += 1)
25     {
26         myServo.write(angle);
27         delay(15);
28     }
```

循环执行，机械臂伺服电机从 170° 至 10° 往复旋转。

```
30     for (angle1 = 10; angle1 <= 170; angle1 += 1)
31     {
32         myServo1.write(angle1);
33         delay(15);
34     }
35
36     for (angle1 = 170; angle1 >= 10; angle1 -= 1)
37     {
38         myServo1.write(angle1);
39         delay(15);
40     }
```

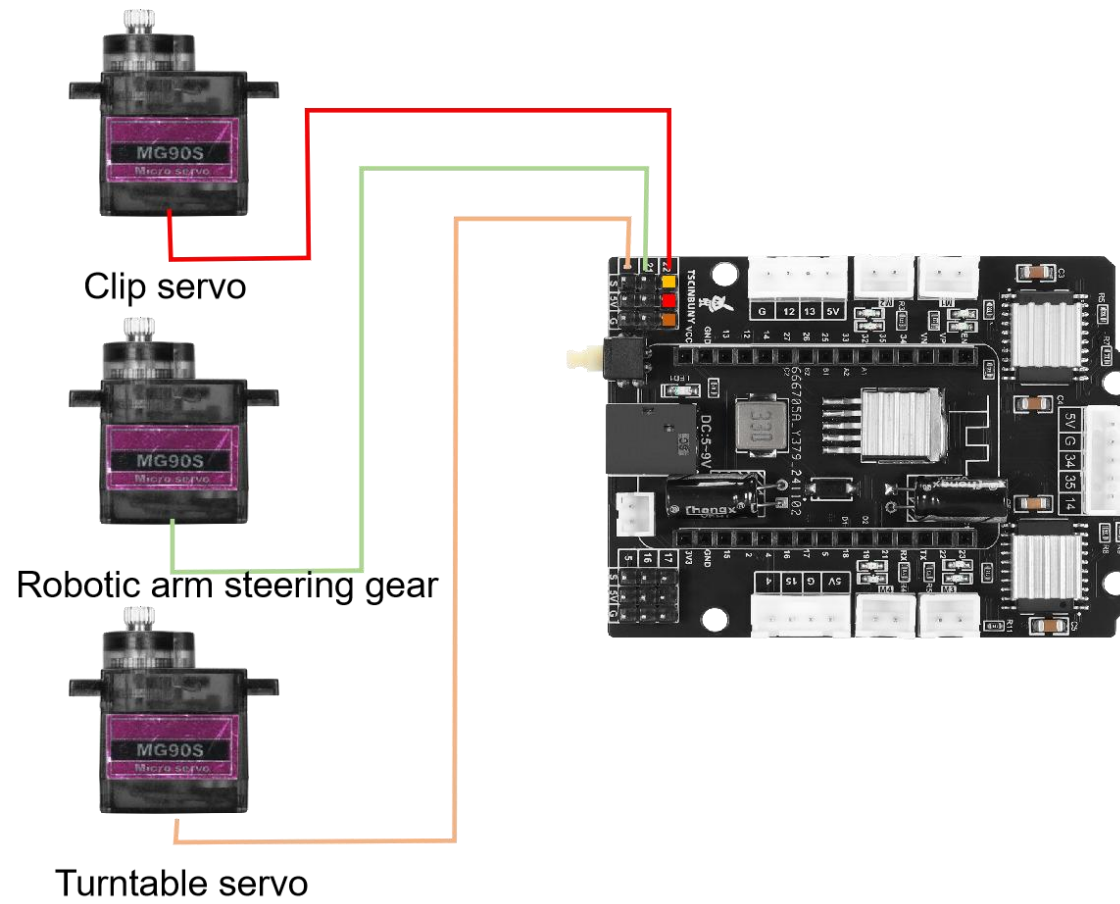



循环执行，夹子伺服电机从 60° 至 0° 往复旋转。

```
42   for (angle2 = 60; angle2 >= 0; angle2 -= 1)
43   {
44       myServo2.write(angle2);
45       delay(15);
46   }
47   for (angle2 = 0; angle2 <= 60; angle2 += 1)
48   {
49       myServo2.write(angle2);
50       delay(15);
51   }
```



舵机接线图





9 综合控制

9.1 课程介绍

在第二课中我们已经安装了 APP，本节课主要理解智能蓝牙控制的编程原理。

9.2 蓝牙综合控制代码分析

蓝牙控制智能小车的基本原理是通过 app 发送字符串到小车，小车接收字符串后，判断字符串类别。

改变小车的运动模式

默认在“自由控制”模式，也就是 `Mode_var = 1`，同时我们设置了另外四种模式。

当蓝牙 APP 按下 OA 按钮，也就是发送了“2”

这时通过代码中“`Switch……case`”判断接收到“1”，并将 2 保存到模式变量中，这时“`Car_mode = 2`”。

当 `Car_mode = 2`，继续检测蓝牙发送的信号



```
case '2':  
    myServo3.write(downServoMidAngle);  
    model_var = 2;  
    pCharacteristic->setValue("model_var = 2");  
    break;
```

在 loop 函数中，当 car_mode 变量为 2 时，执行避障方法。

```
if(model_var == 2){  
    ObstacleAvoidance();  
}
```

同理可以理解其他模式的编程原理，接收到“3”为跟随模式、接收到“4”为循迹模式、接收到“7”为机械臂模式

跟随方法

```
if(UT_distance < 150)  
{  
    moveBackward(motorSpeed);  
}  
  
if(150 <= UT_distance && UT_distance <= 250)  
{  
    stopMoving();  
}  
  
if(UT_distance >= 250)  
{  
    moveForward(motorSpeed);  
}
```



寻迹方法

```
if (Left_Tra_Value < Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value < Black_Line)
{
    myServo3.write(downServoMidAngle);
    moveForward();
}

if (Left_Tra_Value >= Black_Line && Left_Tra_Value > Right_Tra_Value)
{
    myServo3.write(downServoMidAngle + 20);
    TrackLeft();//左循迹
}
```

```
if (Right_Tra_Value >= Black_Line && Right_Tra_Value > Left_Tra_Value)
{
    myServo3.write(downServoMidAngle - 20);
    TrackRight();//右循迹
}

if (Left_Tra_Value >= Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value >= Black_Line)
{
    stopMoving();
}

if (Left_Tra_Value < Black_Line && Center_Tra_Value < Black_Line && Right_Tra_Value < Black_Line)
{
    stopMoving();
}
```



机械臂方法

```
void arm_control(){  
    while(AngleI)  
    {  
        angle1 += 10;  
        if(angle1 > 180)  
        {  
            angle1 =180;  
        }  
        myServo1.write(angle1);  
        delay(200); // 等待舵机到达指定位置  
        Serial.print("Angle1: ");  
        Serial.println(angle1);  
    }  
}
```

至此，已完成了整个教程的学习！